Лабораторная работа: Flask + MongoDB в Docker

Цель работы

Изучить контейнеризацию приложений с помощью Docker и Docker Compose, настроить взаимодействие Flask-приложения с MongoDB, реализовать обработку ошибок подключения и операций с БД.

Задачи

1. Развернуть MongoDB в Docker-контейнере.
2. Написать Flask-приложение с эндпоинтом /hello, который обращается к MongoDB.
3. Реализовать корректную обработку ошибок подключения и запросов.
4. Связать контейнеры через docker-compose с healthcheck и зависимостями.
5. Проверить работу через curl/ браузер.

Создать все файлы согласно структуре.

```
flask-mongo-lab/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── db.py
│   ├── config.py
│   └── requirements.txt
├── mongo-init/
│   └── init.js
├── .env
├── .dockerignore
├── Dockerfile
├── docker-compose.yml
└── README.md
```

Содержимое файла .env

```
MONGO_ROOT_USER=admin
MONGO_ROOT_PASSWORD=secret123
MONGO_DB=hello_db
MONGO_HOST=mongo
MONGO_PORT=27017
FLASK_PORT=5000
FLASK_ENV=production
```

Содержимое файла app/config.py

```
import os
from dotenv import load_dotenv

load_dotenv()


class Config:
    MONGO_USER = os.getenv("MONGO_ROOT_USER", "admin")
    MONGO_PASSWORD = os.getenv("MONGO_ROOT_PASSWORD", "secret123")
    MONGO_HOST = os.getenv("MONGO_HOST", "mongo")
    MONGO_PORT = int(os.getenv("MONGO_PORT", 27017))
    MONGO_DB = os.getenv("MONGO_DB", "hello_db")

    # Таймаут выбора сервера (важно для быстрого падения при
недоступности)
    MONGO_TIMEOUT_MS = 5000

    @property
    def mongo_uri(self) -> str:
        return (
            f"mongodb://{self.MONGO_USER}:{self.MONGO_PASSWORD}"
            f"@{self.MONGO_HOST}:{self.MONGO_PORT}/?authSource=admin"
    )
```

Содержимое файла app/db.py

```
import logging
from typing import Optional

from pymongo import MongoClient
from pymongo.errors import (
    PyMongoError,
    ServerSelectionTimeoutError,
    ConfigurationError,
    ConnectionFailure,
    OperationFailure,
)

from .config import Config

logger = logging.getLogger(__name__)
```

```
class MongoDB:
    """Обёртка над PyMongo с корректной обработкой ошибок."""

    def __init__(self, config: Config):
        self.config = config
        self._client: Optional[MongoClient] = None
        self._db = None

    def connect(self) -> bool:
        try:
            self._client = MongoClient(
                self.config.mongo_uri,
                serverSelectionTimeoutMS=self.config.MONGO_TIMEOUT_MS,
                retryWrites=True,
                retryReads=True,
            )
            # Реальная проверка соединения
            self._client.admin.command("ping")
            self._db = self._client[self.config.MONGO_DB]
            logger.info("Подключение к MongoDB установлено")
            return True

        except ServerSelectionTimeoutError:
            logger.error("MongoDB недоступна (timeout)")
        except ConfigurationError as e:
            logger.error("Ошибка конфигурации MongoDB: %s", e)
        except ConnectionFailure as e:
            logger.error("Ошибка соединения с MongoDB: %s", e)
        except PyMongoError as e:
            logger.error("Ошибка PyMongo: %s", e)

        return False

    def close(self):
        if self._client:
            self._client.close()
            logger.info("Соединение с MongoDB закрыто")

    @property
    def db(self):
        if self._db is None:
            raise RuntimeError("MongoDB не инициализирована")
        return self._db

    def ping(self) -> bool:
        try:
            self._client.admin.command("ping")
```

```
            return True
        except PyMongoError:
            return False


# Глобальный экземпляр (для простоты лабы)
mongo = MongoDB(Config())
```

Содержимое файла app/main.py

```
import logging
from flask import Flask, jsonify
from .config import Config
from .db import mongo
from pymongo.errors import PyMongoError

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)
app = Flask(__name__)
@app.route("/")
@app.route("/hello")
def hello():
    """Возвращает приветствие и текущее время из MongoDB."""
    try:
        # Проверяем соединение
        if not mongo.ping():
            return jsonify({
                "status": "error",
                "message": "MongoDB недоступна",
            }), 503

        collection = mongo.db["greetings"]

        # Атомарно увеличиваем счётчик обращений
```

```
        result = collection.find_one_and_update(
            {"_id": "counter"},
            {"$inc": {"visits": 1}},
            upsert=True,
            return_document=True,
        )

        visits = result.get("visits", 0) if result else 0

        return jsonify({
            "status": "ok",
            "message": "Hello, World!",
            "db": Config.MONGO_DB,
            "visits": visits,
        })
    except OperationFailure as e:
        app.logger.error("Ошибка операции MongoDB: %s", e)
        return jsonify({"status": "error", "message": "Ошибка запроса"}),
500

    except PyMongoError as e:
        app.logger.error("Ошибка PyMongo: %s", e)
        return jsonify({"status": "error", "message": "Ошибка БД"}), 500

    except Exception as e:
        app.logger.exception("Непредвиденная ошибка")
        return jsonify({"status": "error", "message": "Внутренняя
ошибка"}), 500

@app.route("/health")
def health():
    """Healthcheck для Docker."""
    if mongo.ping():
        return jsonify({"status": "healthy"}), 200
    return jsonify({"status": "unhealthy"}), 503
```

```
if __name__ == "__main__":
    # Пытаемся подключиться на старте
    if not mongo.connect():
        app.logger.warning("Не удалось подключиться к MongoDB при старте")

    app.run(host="0.0.0.0", port=5000)
```

Содержимое файла app/requirements.txt

```
Flask
pymongo
python-dotenv
gunicorn
```

Содержимое Dockerfile

```
FROM python:3.12-slim

# Не создавать .pyc и не буферизовать stdout
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# Сначала зависимости — для кеширования слоёв
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Затем код
COPY app/ ./app/

# Непривилегированный пользователь
RUN useradd -m appuser && chown -R appuser /app
USER appuser

EXPOSE 5000

CMD ["gunicorn", "-b", "0.0.0.0:5000", "-w", "2", "app.main:app"]
```

Содержимое файла mongo-init/init.js

```
// Инициализация БД при первом запуске
db = db.getSiblingDB('hello_db');

db.greetings.insertOne({
  _id: 'counter',
  visits: 0,
  createdAt: new Date()
});

print('MongoDB инициализирована: hello_db.greetings');
```

Содержимое файла docker-compose.yml

```
services:
  mongo:
    image: mongo:7.0
    container_name: flask-mongo-lab-db
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}
      MONGO_INITDB_DATABASE: ${MONGO_DB}
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
      - ./mongo-init:/docker-entrypoint-initdb.d:ro
    networks:
      - app-network
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand(©ping©).ok"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s

  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: flask-mongo-lab-app
    restart: unless-stopped
    env_file:
      - .env
    environment:
      MONGO_HOST: mongo
      MONGO_PORT: 27017
    ports:
      - "${FLASK_PORT}:5000"
```

```
    depends_on:
      mongo:
        condition: service_healthy
    networks:
      - app-network
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen(©http://localhost:5000/health©)"]
      interval: 15s
      timeout: 5s
      retries: 3
      start_period: 10s

volumes:
  mongo_data:
    driver: local

networks:
  app-network:
    driver: bridge
```

Содержимое файла .dockerignore

```
__pycache__/
*.pyc
.env
.git/
.venv/
venv/
*.md
.pytest_cache/
```

Содержимое файла README.md

````
# Flask + MongoDB Lab
## Запуск
```bash
cp .env.example .env   # при необходимости
docker compose up --build -d
```
## Проверка

```bash
curl http://localhost:5000/hello
curl http://localhost:5000/health
```
Ожидаемый ответ `/hello`:
```json
{
  "status": "ok",
  "message": "Hello, World!",
  "db": "hello_db",
  "visits": 1
}
```
## Остановка
```bash
docker compose down          # остановить
docker compose down -v       # остановить и удалить volumes
```
## Логи
```bash
docker compose logs -f app
docker compose logs -f mongo
```
````

Ход выполнения лабораторной работы

Шаг 1. Клонирование структуры

Создать все файлы согласно структуре выше.

Шаг 2. Сборка и запуск

```
docker compose up --build -d
```

Шаг 3. Проверка статуса контейнеров

```
docker compose ps
```

Оба контейнера должны иметь статус `Up (healthy)`.

Шаг 4. Проверка работы приложения

```
curl http://localhost:5000/hello
```

Счётчик `visits` должен увеличиваться при каждом запросе.

Шаг 5. Проверка обработки ошибок

Остановить MongoDB и убедиться, что приложение возвращает корректную ошибку:

```
docker compose stop mongo
curl -i http://localhost:5000/hello
# Ожидаем HTTP 503 и {"status":"error","message":"MongoDB недоступна"}
docker compose start mongo
```

Шаг 6. Просмотр данных в БД

```
docker exec -it flask-mongo-lab-db mongosh \
  -u admin -p secret123 --authenticationDatabase admin \
  --eval "db.getSiblingDB('hello_db').greetings.find().pretty()"
```

Управление жизненным циклом

Действие	docker run	docker compose
Запустить	docker run ...	docker compose up -d
Остановить	docker stop app mongo	docker compose stop
Удалить	docker rm app mongo	docker compose down
Удалить с volumes	docker volume rm mongo_data	docker compose down -v
Пересобрать	docker build ... && docker run ...	docker compose up --build
Логи всех	docker logs app && docker logs mongo	docker compose logs -f
Зайти в контейнер	docker exec -it app bash	docker compose exec app bash

Контрольные вопросы

1. Зачем нужен healthcheck в docker-compose.yml? Что произойдёт без него?
2. Почему используется depends_on: condition: service_healthy, а не просто depends_on?
3. Какие ошибки PyMongo обрабатываются в db.py и почему именно они?
4. Что произойдёт, если не указать serverSelectionTimeoutMS?
5. Почему в Dockerfile создаётся непривилегированный пользователь?
6. В чём разница между restart: always и restart: unless-stopped?
7. Зачем нужен отдельный эндпоинт /health?
8. Как volumes обеспечивают сохранность данных MongoDB между перезапусками?